

Лабораторная работа №1: Потоки ядра в xv6

Оглавление

1. Процесс vs поток	2
1.1. Зачем нужны потоки	2
1.2. Потоки в реальных ОС: Linux и Windows	4
1.3. Что выберем для xv6	5
2. Как устроен процесс	6
2.1. Общая структура	6
2.2. Разбор fork(): отправная точка для clone	9
3. Дизайн системного вызова clone	12
4. Завершение потока: exit, ZOMBIE и вопрос «кто чистит»	15
4.1. Как это уже устроено в xv6: exit + wait	16
4.2. Ожидание потока: thread_join	18
5. Замер времени в xv6	20
6. Два предупреждения перед практикой	23
6.1. Планировщик не обещает ничего	23
6.2. Гейзенбаги: почему многопоточные ошибки не ловятся привычными способами	23
7. Практическая часть	25
Шаг 1. Инфраструктура замера времени	25
Шаг 2. Скелет системного вызова clone	27
Сверка с эталоном: те же эксперименты на Linux	28
6. Задания	31
6.1. Обязательная часть	31
6.2. Дополнительное задание	31
7. Контрольные вопросы	31

1. Процесс vs поток

1.1. Зачем нужны потоки

Программа, выполняющаяся в системе, — это процесс: у него есть собственная память, открытые файлы и как минимум один поток выполнения. Пока поток один, программа делает ровно одну вещь в каждый момент времени. Если нужно обрабатывать несколько задач одновременно — обслуживать нескольких клиентов, считать на нескольких ядрах процессора, не замораживать интерфейс на время долгой операции — нужны несколько потоков выполнения.

Исторически первым способом получить параллелизм был `fork()`: создать копию процесса. Это работает, но у копии есть два фундаментальных неудобства:

1. **Дорого создавать.** Копирование адресного пространства — даже ленивое, через `copy-on-write` — требует дублирования таблиц страниц и последующих `page fault`'ов.
2. **Дорого обмениваться данными.** У копии своя память: чтобы два процесса увидели общие данные, нужны специальные механизмы (`pipe`, разделяемая память), каждый — с накладными расходами и собственным API.

Поток решает обе проблемы радикально: это ещё одна единица выполнения внутри того же процесса. Общая память достаётся бесплатно — она просто общая.

Что такое поток с точки зрения ядра

Чтобы код мог выполняться, ядру нужно хранить для него:

- **контекст процессора** — значения регистров (включая счётчик команд `pc` и указатель стека `sp`) в момент, когда поток был приостановлен;
- **стек** — память для локальных переменных и адресов возврата;
- **учётную информацию** — состояние (выполняется / готов / спит), идентификатор, приоритет.

А чтобы код мог работать с данными, нужны:

- **адресное пространство** — таблица страниц, отображающая виртуальные адреса в физические;
- **ресурсы** — открытые файлы, текущий каталог и т.п.

Ключевое наблюдение: первая группа описывает *выполнение*, вторая — *окружение*. Процесс в классическом Unix склеивает обе группы в одну сущность. Поток — это результат их разделения:

Процесс — контейнер ресурсов: адресное пространство + открытые файлы.

Поток — единица выполнения внутри контейнера: регистры + стек + состояние.

Один процесс может содержать много потоков. Все они видят одну и ту же память и одни и те же файловые дескрипторы, но у каждого — свой стек и свой набор регистров.

Таблица: что общее, что своё

Ресурс	У потоков одного процесса	Почему именно так
Адресное пространство (код, глобальные данные, куча)	общее	в этом смысл потоков: обмен данными через обычные переменные
Открытые файлы (дескрипторы)	общие	офп в одном потоке виден в другом
Стек	свой	у каждого потока своя цепочка вызовов и свои локальные переменные
Регистры (pc, sp, ...)	свои	каждый поток выполняется в своей точке программы
Состояние (RUNNING/SLEEPING/...)	своё	один поток может спать, пока другой работает
Идентификатор	свой (tid)	ядру нужно различать потоки при планировании

Обрати внимание на следствие первой строки: раз куча и глобальные переменные общие, то любая запись одного потока немедленно видна остальным — без каких-либо действий со стороны программиста. Это и главное достоинство потоков, и главный источник проблем: одновременный доступ к общим данным без согласования приводит к гонкам (races), которые мы воспроизведём в экспериментальной части.

Стек: почему он обязан быть своим

Единственный крупный ресурс, который нельзя разделить, — стек. Причина механическая: стек хранит цепочку вызовов. Если поток А выполняет `main() → foo() → bar()`, а поток В — `main() → baz()`, их цепочки различны и должны храниться раздельно; один указатель `sp` на двоих невозможен.

При этом важно понимать: раздельность стеков — это соглашение, а не защита. Все стеки лежат в общем адресном пространстве, и поток А физически может читать и писать стек потока В (например, получив указатель на его локальную переменную). Иногда это используют сознательно, но чаще это источник ошибок: указатель на локальную переменную потока становится недействительным, когда поток завершается или выходит из функции.

Адресное пространство процесса (общее для всех потоков):

+-----+ код (text)	← общий, все потоки исполняют одни инструкции
+-----+ глобальные данные (data)	← общие
+-----+ куча (heap)	← общая, malloc виден всем
+-----+ стек потока 3	← "свой" для потока 3
+-----+ стек потока 2	← "свой" для потока 2
+-----+ стек потока 1 (main)	← "свой" для потока 1
+-----+	

Модель 1:1

Остаётся вопрос: кто планирует потоки — ядро или библиотека в userspace? Возможны три модели:

- **1:1** — каждый поток пользователя есть отдельная задача ядра; планирует ядро;
- **M:1** — все потоки процесса живут в одной задаче ядра; планирует библиотека; блокирующий syscall одного потока останавливает все;
- **M:N** — гибрид: M пользовательских потоков на N задачах ядра; максимально гибко и максимально сложно.

Linux, Windows и почти все современные системы используют 1:1: она проста, честно использует многоядерность, и блокировка одного потока не трогает остальные. Мы в этой лабораторной делаем то же самое: наш поток в xv6 будет полноценной записью в таблице процессов ядра, которую планировщик xv6 обрабатывает наравне с обычными процессами. (Модель M:N не умерла — горутины Go устроены именно так, и к идее планирования в userspace мы вернёмся в конце курса, в лабораторных про корутины.)

Итого, задача лабораторной формулируется в одном предложении:

Научить xv6 создавать запись в таблице процессов, которая разделяет адресное пространство с родителем, но имеет собственные стек, регистры и идентификатор — и дать userspace-интерфейс `thread_create/thread_join` к этому механизму.

1.2. Потоки в реальных ОС: Linux и Windows

Раздел 1.1 описал поток как идею: единица выполнения внутри контейнера ресурсов. Но идею можно воплотить по-разному, и две главные ОС современности сделали это почти противоположными способами. Полезно увидеть оба — тогда наше решение для xv6 будет осознанным выбором, а не единственным известным вариантом.

Windows NT: поток как отдельная сущность с рождения

Ядро NT проектировалось в конце 1980-х, когда потоки уже были осознанной необходимостью, — и разделение «контейнер/выполнение» заложено в него буквально:

- **EPROCESS** — объект ядра «процесс»: адресное пространство, таблица описателей (handles), учётные данные безопасности. Процесс в NT не выполняется — это только контейнер.
- **ETHREAD/KTHREAD** — объект ядра «поток»: контекст регистров, два стека (пользовательский и ядерный), приоритет. Планировщик NT работает только с потоками; процессы для него невидимы.

Создание процесса (`CreateProcess`) всегда создаёт и его первый поток; дальнейшие добавляются через `CreateThread`. Это разные объекты с разными системными вызовами — никакого родства между «создать процесс» и «создать поток» в API нет.

Схема из раздела 1.1 — «процесс = контейнер, поток = единица выполнения» — это буквально архитектура NT.

Linux: поток как процесс с общими ресурсами

Unix родился в 1970-х без потоков — была только пара `fork()/exec()`. Когда потоки понадобились, перед Linux встал выбор: вводить отдельную сущность (путь NT) или обобщить то, что есть. Linux выбрал второе, и решение получилось изящным.

В ядре Linux нет структуры «поток» и нет структуры «процесс» — есть одна `task_struct` («задача»), и есть системный вызов `clone()`, который создаёт новую задачу, принимая флаги: что разделить с родителем, а что скопировать:

```
clone(fn, stack, flags, arg);

// флаги (основные):
CLONE_VM      // общее адресное пространство
CLONE_FILES   // общая таблица открытых файлов
CLONE_FS      // общий текущий каталог
CLONE_SIGHAND // общие обработчики сигналов
CLONE_THREAD  // общий PID (та же "группа потоков")
```

И тогда:

- `clone()` без флагов деления $\approx$ `fork()`: всё копируется, получаем классический процесс;
- `clone()` со всеми флагами выше — получаем поток: та же память, те же файлы, тот же PID снаружи.

«Процесс» и «поток» в Linux — не разные сущности, а две точки на шкале степени деления ресурсов. Библиотека `pthread` (`glibc/NPTL`) — это `userspace`-обёртка, которая выделяет стек через `mmap` и вызывает `clone()` с «поточным» набором флагов. Между этими точками существуют и промежуточные комбинации (например, общая память без общих файлов) — на этой гибкости построены контейнеры, но это отдельная история.

Для различения задач у Linux два идентификатора: TID — уникальный номер задачи, и TGID (`thread group id`) — общий номер «процесса», который возвращает `getpid()` из любого потока. У главного потока `TID == TGID`.

```
Процесс глазами пользователя (PID=1000)
├─ задача TID=1000, TGID=1000  ← главный поток
├─ задача TID=1001, TGID=1000  ← pthread_create
└─ задача TID=1002, TGID=1000  ← pthread_create
```

Сравнение и общее ядро

	Windows NT	Linux
Сущности в ядре	две: <code>EPROCESS</code> и <code>ETHREAD</code>	одна: <code>task_struct</code>
Поток создаётся	<code>CreateThread</code> — отдельный механизм	<code>clone()</code> — тот же механизм, что для процессов
«Что разделять»	зашиито в архитектуру	выбирается флагами на каждый <code>clone</code>
ID	поток: TID; процесс: PID	TID; TGID играет роль PID
Планировщик видит	потоки	задачи

Различие глубокое, но заметь: модель у обеих — 1:1 (раздел 1.1), и планировщик в обоих случаях оперирует единицами выполнения, а «процесс» — учётной оболочкой. Расходятся они в том, где проведена граница между сущностями, — и это в значительной мере след истории: NT строилась с потоками с нуля, Linux дорастил потоки поверх Unix-наследия.

1.3. Что выберем для xv6

`xv6` — маленький Unix, и его положение повторяет Linux начала 1990-х: есть `struct proc`, есть `fork()`, потоков нет. Естественный путь для нас — путь Linux: не вводить новую сущность, а добавить системный вызов `clone`, создающий запись в той же таблице процессов, но с общим адресным пространством. Наши «потоки» будут обычными

записями `struct proc`, которые планировщик `xv6` обрабатывает наравне с процессами, — плюс новое поле `tgid`, как в `Linux`.

Этот выбор не только дешевле по объёму изменений — он воспроизводит реальную эволюцию реальной ОС, и после лабораторной устройство `pthread` в `Linux` будет для тебя не справочным фактом, а вещью, которую ты один раз построил своими руками.

Теперь посмотрим на исходный материал — что представляет собой `struct proc` в `xv6` (раздел 1.3).

2. Как устроен процесс

2.1. Общая структура

Вся информация о процессе в `xv6` хранится в одной структуре — `struct proc` (файл `kernel/proc.h`). Таблица процессов — просто статический массив таких структур:

```
// kernel/proc.c
struct proc proc[NPROC]; // NPROC = 64
```

Никакого динамического выделения: максимум 64 процесса, свободная запись ищется перебором. Для учебного ядра это осознанное упрощение, и нам оно на руку — все процессы (и будущие потоки) всегда можно обойти циклом.

Разберём поля структуры, группируя их по смыслу — в терминах раздела 1.1: что здесь «контейнер ресурсов», а что «единица выполнения».

Структура целиком

```
// kernel/proc.h
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    int xstate;           // Exit status to be returned to parent's wait
    int pid;              // Process ID

    // wait_lock must be held when using this:
    struct proc *parent; // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // swtch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];          // Process name (debugging)
};
```

Группа 1: ресурсы (то, что будет у потоков общее)

pagetable — корень таблицы страниц процесса. Это и есть «адресное пространство»: отображение виртуальных адресов пользователя в физическую память. Когда `fork()` создаёт ребёнка, он строит новую таблицу и копирует в неё всё содержимое родительской (`uvmfsor`). Наш `clone` вместо этого запишет сюда тот же указатель, что у родителя, — одна строка кода, в которой сосредоточена половина смысла лабораторной. Все обращения к памяти обоих потоков пойдут через одну таблицу — общая память достигнута.

sz — размер пользовательской памяти процесса в байтах. **xv6** держит образ процесса непрерывным: от адреса 0 до **sz**. Системный вызов **sbrk** увеличивает **sz**, доращивая кучу. Обрати внимание на ловушку, которая выстрелит позже: если **pagetable** станет общей, то **sz** у двух записей **proc** описывает одну и ту же память — а хранится в двух экземплярах. Один поток сделает **sbrk**, его **sz** изменится, а у второго останется старым. Это реальная проблема согласованности; в базовой версии лабораторной мы просто запретим **sbrk** из потоков, а её честное решение — дополнительное задание.

ofile[NOFILE] — таблица открытых файлов: указатели на общесистемные **struct file**. При **fork** массив копируется (с увеличением счётчиков ссылок). Тонкость для нашего **clone**: скопировать массив — значит зафиксировать его состояние на момент создания; если после этого один поток откроет файл, другой этого не увидит. Настоящие потоки **Linux** разделяют саму таблицу (флаг **CLONE_FILES**). В базовой версии мы согласимся на копию (как у **fork**) и явно зафиксируем это как отклонение от семантики потоков.

cwd — текущий каталог. Та же история, что с файлами.

Группа 2: выполнение (то, что у потока своё)

kstack — стек ядра. Важно не путать с пользовательским стеком! У каждого процесса в **xv6** два стека:

- *пользовательский* — лежит в адресном пространстве процесса, на нём выполняется код программы;
- *ядерный* (**kstack**) — отдельная страница в памяти ядра; когда процесс входит в ядро (**syscall**, прерывание), выполнение идёт на этом стеке.

Потоку нужны свои оба: пользовательский — потому что своя цепочка вызовов в программе (раздел 1.1), ядерный — потому что два потока могут одновременно находиться внутри ядра (например, оба сделали **syscall** на разных **CPU**). **allocproc** выделяет **kstack** автоматически — здесь нам ничего менять не придётся. А вот пользовательский стек для нового потока ядро **xv6** не выделяет — им займёмся мы сами, и это отдельный проектный вопрос (раздел 1.4).

trapframe — страница, куда при входе в ядро (**trap**) сохраняются все пользовательские регистры: **pc** (в поле **epc**), **sp**, аргументные регистры **a0–a7** и остальные. При возврате из ядра регистры восстанавливаются из **trapframe**. Для нас это главный рычаг управления: чтобы новый поток начал выполнение с функции **fn**, со стеком **stack** и аргументом **arg**, достаточно записать в его **trapframe**:

```
trapframe->epc = fn;           // счётчик команд: откуда начать
trapframe->sp   = ...;         // указатель стека: вершина выделенного стека
trapframe->a0   = arg;         // первый аргумент функции (соглашение RISC-V)
```

— и когда планировщик впервые «вернёт» этот поток в **user mode**, он окажется в начале **fn(arg)**. Никакой магии: точка входа потока — это просто три поля **trapframe**.

context — сохранённые **callee-saved** регистры ядерного контекста. Используется функцией **swtch** при переключении планировщиком: **trapframe** хранит «где процесс был в **userspace**», **context** — «где он был внутри ядра». Разделение труда этих двух структур стоит проговорить на защите. Для **clone** здесь менять ничего не нужно.

state — состояние:

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

Жизненный цикл: allocproc берёт UNUSED запись → RUNNABLE → планировщик делает RUNNING → при завершении ZOMBIE до тех пор, пока родитель не сделает wait. Каждый наш поток проживёт этот же цикл; что означает ZOMBIE для потока и кто его «дочищает» — разберём в 1.5.

chan — «канал» сна: если процесс в состоянии SLEEPING, здесь лежит адрес, на котором он уснул (sleep(chan, lk)), а wakeup(chan) будит всех, кто спит на этом адресе. Поскольку sleep()/wakeup() работают в ядре, речь идёт об адресах, доступных ядру, а не о произвольных пользовательских адресах. Этот механизм — основа нашего thread_join (раздел 1.6), а в лабораторной №2 из него вырастет futex.

pid — идентификатор. Для потоков нам понадобится различать: уникальный номер записи (tid — им станет обычный pid, который выдаёт allocproc) и общий номер «процесса-контейнера» (tgid — новое поле, которое мы добавим и которое у всех потоков одного процесса будет совпадать).

Группа 3: служебные поля

lock — спинлок, защищающий изменяемые поля (state, chan, killed, ...). Таблица процессов — разделяемая структура: к записи может одновременно обращаться сам процесс, планировщик на другом CPU и родитель, делающий wait. Правило xv6: менять state можно только под p->lock. Наш код clone/join обязан соблюдать ту же дисциплину — иначе мы внесём гонки в ядро, которое учим бороться с гонками.

parent + xstate — кто родитель и с каким кодом процесс завершился: механика wait(). thread_join будет использовать её почти без изменений.

killed, name — флаг принудительного завершения и имя для отладки.

Сводная карта: что делает clone с каждым полем

Поле	fork()	наш clone()
pagetable	новая + uvmcopy (копия)	указатель родителя (общая)
sz	копия значения	копия значения (+ известная проблема)
ofile[], cwd	копия (с ref++)	копия (упрощение)
kstack	свой (даёт allocproc)	свой (даёт allocproc)
пользовательский стек	копируется в составе памяти	выделяет вызывающий, передаёт в clone
trapframe	копия родительского (ребёнок «возвращается из fork»)	свой: epc=fn, sp=stack, a0=arg
context, state, chan	стандартный жизненный цикл	стандартный жизненный цикл
pid	новый	новый (это tid) + новое поле tgid — общий

Две выделенные строки — pagetable и trapframe — и есть вся суть clone. Остальное — аккуратность вокруг них: не освободить общую pagetable при завершении (1.5), корректно дожидаться потока (1.6).

Чтобы понять, куда именно вносить изменения, в следующем разделе построчно разберём fork() — функцию, из которой наш clone вырастет.

2.2. Разбор fork(): отправная точка для clone

Наш clone — это fork с тремя изменёнными местами. Поэтому, прежде чем писать своё, разберём чужое построчно. Вот fork() из kernel/proc.c ванильного xv6-riscv целиком:

```
// Create a new process, copying the parent.
// Sets up child kernel stack to return as if from fork() system call.
int
fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;

    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;

    // increment reference counts on open file descriptors.
    for(i = 0; i < NOFILE; i++)
        if(p->ofile[i])
            np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);

    safestrcpy(np->name, p->name, sizeof(p->name));

    pid = np->pid;

    release(&np->lock);

    acquire(&wait_lock);
    np->parent = p;
    release(&wait_lock);

    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);

    return pid;
}
```

Пройдём по шагам.

Шаг 1: allocproc() — найти и подготовить запись

```
if((np = allocproc()) == 0){
    return -1;
}
```

allocproc (там же, в proc.c) делает четыре вещи:

1. перебирает массив proc[NPROC] в поисках записи в состоянии UNUSED;
2. выдаёт новый pid (глобальный счётчик под спинлоком);
3. выделяет страницу под trapframe;
4. создаёт пустую pagetable процесса (proc_pagetable) — в ней пока отображены только trampoline и trapframe, пользовательской памяти нет;

5. настраивает context так, чтобы первый запуск процесса начался с функции forkret (служебный трамплин ядра, который завершает переход в user mode).

Важная деталь: allocproc возвращает запись с захваченным np->lock — поэтому дальше по коду fork в какой-то момент делает release(&np->lock). Это дисциплина блокировок xv6: пока запись не готова, никто (планировщик на другом CPU!) не должен увидеть её в промежуточном состоянии.

Для clone этот шаг подходит почти целиком. «Почти» — потому что пункт 4 создаёт новую pagetable, а нам она не нужна: мы собираемся использовать родительскую. Возникает выбор: модифицировать allocproc (параметром) или дать ему создать таблицу и тут же освободить, подставив родительскую. Второй путь чуть расточителен, но не трогает общий код — для лабораторной предпочтём его; желающие могут сделать аккуратнее.

Шаг 2: uvmcopy — копирование памяти. Сердце fork и главное отличие clone

```
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
np->sz = p->sz;
```

uvmcopy (kernel/vm.c) идёт по родительской pagetable от адреса 0 до sz и для **каждой** страницы: выделяет новую физическую страницу, копирует в неё содержимое (memmove), отображает в таблицу ребёнка. После этого у ребёнка — независимая копия всей пользовательской памяти.

Здесь же видна цена fork: время работы линейно по объёму памяти процесса. (В настоящем Linux эту цену снижает copy-on-write — страницы разделяются до первой записи; в xv6 копирование честное и немедленное. Наш эксперимент Э3 покажет разницу clone/fork численно.)

Для clone весь этот блок заменяется на:

```
// вместо uvmcopy:
np->pagetable = p->pagetable; // общее адресное пространство
np->sz = p->sz;
```

— плюс освобождение таблицы, созданной allocproc, и пометка «таблица общая» (понадобится на завершении — раздел 1.6). Одна строка вместо копирования всей памяти: вот почему создание потока дешевле создания процесса.

Шаг 3: trapframe — ребёнок «возвращается из fork»

```
// copy saved user registers.
*(np->trapframe) = *(p->trapframe);

// Cause fork to return 0 in the child.
np->trapframe->a0 = 0;
```

Изящное место. В trapframe родителя сейчас лежит его состояние на момент входа в syscall: ерс указывает на инструкцию после еcall fork, sp — на его стек, все регистры — как были. Скопировав trapframe целиком, fork делает так, что ребёнок при первом выходе в user mode окажется в той же точке программы — как будто он тоже только что вернулся из fork.

Единственная правка — a0 = 0: по соглашению RISC-V в a0 лежит возвращаемое значение. Родителю syscall вернёт pid ребёнка, ребёнку — 0. Так работает знаменитое «fork возвращается дважды».

Clone здесь расходится с fork принципиально. Наш поток не должен «возвращаться из clone» — он должен *начать* выполнение функции `fn(arg)` на новом стеке. Значит, вместо копирования родительского контекста мы конструируем свой:

```
np->trapframe->epc = (uint64)fn;           // начать с fn
np->trapframe->sp  = (uint64)stack + STACK_SIZE; // вершина нового стека
np->trapframe->a0  = (uint64)arg;           // аргумент fn — в a0 по ABI
```

(Скопировать родительский `trapframe` как основу — можно и даже полезно: остальные регистры получают осмысленные значения; но эти три поля перезаписываются обязательно. Тонкости — выравнивание `sp`, что будет при `return` из `fn` — обсудим в 1.5.)

Шаг 4: файлы и каталог

```
for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = idup(p->cwd);
```

Каждый ненулевой указатель массива `ofile` дублируется через `filedup` — это не копирование файла, а инкремент счётчика ссылок на общесистемную `struct file`. Родитель и ребёнок после `fork` указывают на те же открытые файлы (общая позиция чтения!), но массивы у них независимые: `open/close` после `fork` друг на друга не влияют.

Для `clone` оставляем как есть — с оговоркой из раздела 1.3: настоящие потоки Linux разделяют саму таблицу (`CLONE_FILES`), у нас будет копия-снимок. Для наших тестов это несущественно, но в отчёте отличие нужно зафиксировать.

Шаг 5: родство и запуск

```
pid = np->pid;
release(&np->lock);
acquire(&wait_lock);
np->parent = p;
release(&wait_lock);

acquire(&np->lock);
np->state = RUNNABLE;
release(&np->lock);

return pid;
```

Три аккуратных действия под правильными замками:

- `parent = p` — под `wait_lock`: это поле читает `wait()/exit()`, у него свой замок;
- `state = RUNNABLE` — под `np->lock`: с этого момента запись видна планировщику, и новый процесс может быть запущен на другом CPU буквально в следующий такт. Поэтому `RUNNABLE` выставляется строго последним — всё остальное к этому моменту обязано быть готово;
- вернуть `pid` родителю.

Танец с `release/acquire` вокруг `wait_lock` не случаен: порядок захвата замков в xv6 фиксирован (во избежание дедлока ядра), и наш `clone` обязан его повторить в точности. Проще всего — скопировать этот хвост `fork` без изменений, поменяв лишь то, что `pid` мы вернём как `tid`, и добавив установку `tgid` (раздел 1.5).

Сводка: карта изменений `fork` → `clone`

Фрагмент <code>fork</code>	Судьба в <code>clone</code>
<code>allocproc()</code>	без изменений (+ освободить лишнюю <code>pagetable</code>)
<code>uvmcopy(...)</code>	заменить: <code>np->pagetable = p->pagetable</code> , флаг «общая»

np->sz = p->sz	без изменений (с известной оговоркой про sbrk)
*(np->trapframe) = *(p->trapframe)	можно оставить как основу...
np->trapframe->a0 = 0	заменить: epс = fn, sp = вершина стека, a0 = arg
ofile/cwd	без изменений (снимок; зафиксировать отличие от CLONE_FILES)
name, parent, RUNNABLE, замки	без изменений, порядок замков — свято
return pid	вернуть tid; установить np->tgid = p->tgid

Итого содержательных правок две — pagetable и trapframe, — ровно как обещала сводная карта раздела 1.3. Всё остальное — инженерная аккуратность вокруг них. Следующий раздел — проектирование интерфейса clone(fn, arg, stack): сигнатура, ответственность за стек, судьба возврата из fn.

3. Дизайн системного вызова clone

Прежде чем писать код, зафиксируем интерфейс. У настоящего Linux clone принимает функцию, стек, флаги и аргумент. Нам флаги не нужны — набор «что разделять» у нас один, поточный. Остаётся:

```
int clone(void (*fn)(void *), void *arg, void *stack);
```

// возврат: tid нового потока, либо -1 при ошибке

Сигнатура выглядит просто, но за каждым параметром — проектное решение, которое надо принять осознанно. Разберём их по очереди.

Решение 1: стек выделяет вызывающий

Почему стек — параметр? Казалось бы, ядру виднее: пусть само выделит страницы и вернёт готовый поток. Но подумаем, где ядро должно их выделить.

После fork вопроса о стеке нет: ребёнок получает копию всей памяти, и его стек лежит по тем же адресам, что у родителя, только в другой таблице страниц. При clone таблица страниц общая, поэтому новому потоку нужен свой участок в той же памяти, где уже лежат код, данные, куча и стек главного потока. Раскладкой этой памяти распоряжается пользовательский код: куча растёт через sbrk под управлением malloc, и библиотека знает, что где занято. Если бы ядро само вырезало кусок под стек, оно действовало бы за спиной у malloc. Поэтому ядро даёт механизм («запусти контекст с таким sp»), а политику («где взять память, сколько, когда вернуть») оставляет пользователю. В Linux так же: clone принимает указатель на стек, а сам стек glibc получает через mmap.

Пользовательская память в xv6 — это сплошной отрезок от 0 до p->sz, и ядро умеет только двигать его верхнюю границу (growproc). Представим, что ядро выделяет стек потока, увеличивая sz. Потом malloc ещё раз зовёт sbrk, и его арена ложится поверх. Когда поток завершится, вернуть его стек ядру уже нельзя: он в середине отрезка, а уменьшать sz можно только с конца. Память утечёт навсегда. А malloc/free как раз умеют переиспользовать дыры в середине кучи. Поэтому стек, взятый через malloc, после free просто достанется следующему thread_create. Это маленький аналог кэша стеков в glibc.

А вот в Linux ситуация обстоит чуть иначе и причины выделения стека из пространства пользователя немного другие.

Первая — glibc кладёт в этот участок памяти не только стек. `allocate_stack` делает один `mmap` и размечает его так: внизу `guard page` (`PROT_NONE`), над ней стек, а в самой верхушке TLS-блок и `struct pthread`, тот самый TCB с `tid`, `result`, `joinid`. Ядро ничего не знает об этой раскладке: TCB — сущность `libc`, у Go или у `musl` она другая. Выделяя ядро только стек, `libc` всё равно пришлось бы отдельно выделять память под TCB, и управлять пришлось бы двумя кусками вместо одного.

Вторая — время жизни. Если бы ядро само освобождало стек при смерти потока, вместе с ним исчез бы и `struct pthread`, а значит, и `pd->result`, который `pthread_join` должен прочесть уже после смерти потока. `libc` нужно, чтобы память пережила поток до момента `join` (или до переиспользования, если поток `detached`). Ядро даёт ровно один сигнал — обнуляет `tid` через `CLONE_CHILD_CLEARTID`, а решение, когда освобождать, оставляет `libc`.

Третья — производительность. `mmap/munmap` на каждый поток означают системные вызовы, сброс TLB и заново `page fault`'ы на первых обращениях к стеку. Поэтому у glibc есть кэш стеков: умерший поток не вызывает `munmap`, его память целиком (со всей разметкой) достаётся следующему `pthread_create`. Такую политику может вести только тот, кто знает, что это за память.

Четвёртая — разным рантаймам нужно разное. `pthread_attr_setstack` позволяет передать свой, заранее выделенный буфер (например, в статическом массиве или в `hugepage`). Go выделяет стеки системных потоков своим аллокатором, а горутины вообще живут на маленьких стеках в куче, которые рантайм сам копирует при росте. Если ядро навязывало бы одну стратегию, все эти схемы стали бы невозможны.

Поэтому `clone` в Linux устроен как чистый механизм: «дай новому контексту вот такой `sp`» (в `clone3` передаются `stack` и `stack_size`). Единственный стек, которым ядро управляет само, — стек главного потока: его создаёт `execve`, и это VMA с флагом `VM_GROWSDOWN`, который автоматически растёт вниз при `page fault`'е, до `RLIMIT_STACK`. Стеки остальных потоков фиксированного размера (по умолчанию тоже берётся из `RLIMIT_STACK`, обычно 8 МБ) и не растут: вылетел за них, упёрся в `guard page`, получил `SIGSEGV`.

Переложив выделение на вызывающего, мы получаем:

- ядро не трогает `layout` памяти — `clone` остаётся хирургически маленьким;
- пользователь волен взять стек откуда угодно: `malloc`, статический массив, кусок разделяемого буфера;
- освобождение — симметрично выделению, забота пользователя (в `thread_join`).

Цена решения: ядро не может проверить, что стек «хороший» (достаточного размера, не пересекается с чужим). Передашь буфер в 64 байта — получишь загадочную порчу памяти. Это осознанный компромисс учебной реализации.

Решение 2: указатель на вершину, выравнивание

В `trapframe->sp` кладём не `stack`, а `stack + STACK_SIZE`: стек RISC-V растёт вниз, от старших адресов к младшим. Передать в `sp` начало буфера — значит подарить потоку ноль байт: первый же `push` уйдёт в память *перед* буфером.

Вторая деталь: ABI RISC-V требует выравнивания `sp` по 16 байт. `malloc` в `xv6` возвращает адреса, выровненные по 16, и если `STACK_SIZE` кратен 16 — всё сходится автоматически. Но полагаться на удачу не будем: в `clone` принудительно округлим `sp` вниз:

```
uint64 sp = (uint64)stack + STACK_SIZE;
sp &= ~0xf; // выравнивание по 16
np->trapframe->sp = sp;
```

Невыровненный стек — источник особо неприятных багов: часть кода работает, а падает какая-нибудь функция глубоко внутри `printf`, использующая инструкции с требованием выравнивания.

Решение 3: судьба `return` из `fn` — проблема, которую отложим

Вопрос на засыпку: поток начал выполнение с `fn(arg)`. Что произойдёт, когда `fn` сделает `return`?

Инструкция `ret` на RISC-V — это переход по адресу в регистре `ra` (return address). Обычно `ra` заполняет инструкция вызова `jal` — но нашу `fn` никто не вызывал: она началась «из ниоткуда», по значению `pc` из `trapframe`. В `ra` окажется мусор, скопированный из родительского `trapframe`, — и `return` унесёт поток по случайному адресу. В лучшем случае — мгновенная ошибка страницы, в худшем — тихое выполнение случайного кода.

Инженерные решения существуют — например, положить в `ra` адрес функции-финализатора, которая вызовет `exit`. Но в `userspace` `xv6` нет удобного места для такого трамплина без дополнительных ухищрений. Для базовой версии принимаем контрактное решение:

Функция потока обязана завершаться явным вызовом `exit(0)`. `return` из неё — неопределённое поведение.

Это честное упрощение, у него есть прецедент: ровно так же ведёт себя голый `clone` в Linux (`segfault` при возврате), а трамплин с автоматическим `exit` добавляет уже `pthread`. Наш трамплин — дополнительное задание: тем, кто возьмётся, подсказка — `ra` тоже лежит в `trapframe`.

Решение 4: `tgid` и идентичность потока

Добавляем в `struct proc` одно поле:

```
int tgid; // thread group id: общий для всех потоков процесса
```

Правила заполнения — как в Linux:

- `fork`: новый процесс — новая группа: `np->tgid = np->pid`;
- `clone`: тот же процесс: `np->tgid = p->tgid`.

У главного потока `tgid == pid`, у остальных `pid` (наш `tid`) свой, `tgid` общий. Менять `getpid()` так, чтобы он возвращал `tgid`, для лабораторной необязательно — но добавить `syscall gettid()` (вернуть `p->pid`) полезно для отладочной печати в тестах.

Зачем `tgid` нужен практически, если планировщику всё равно? Он отвечает на вопрос «кто последний»: когда завершается поток, надо решить, освобождать ли общую `pagetable` — а для этого понять, остались ли ещё живые потоки этой группы. Перебор таблицы процессов с фильтром по `tgid` — ровно этот механизм (детали — раздел 1.6).

Решение 5: чего мы сознательно НЕ делаем

Список упрощений, зафиксированный явно, — он же список тем для «звёздочек» и защиты:

Упрощение	Что было бы по-честному
-----------	-------------------------

sbrk из потока не поддержан (общий sz в N копиях)	общий sz в разделяемой структуре под замком — доп. задание
ofile/cwd — снимок на момент clone	разделяемая таблица файлов (CLONE_FILES)
return из fn — UB	трамплин через подмену ra — доп. задание
стек не проверяется ядром	guard page, проверка диапазона
kill по tid убивает один поток, группа живёт	в Linux сигнал видит группу целиком
никакого TLS	отдельный регистр tp + место в стеке

Итог: спецификация

```
// Системный вызов.
// Создаёт поток: новую запись таблицы процессов, разделяющую
// адресное пространство (pagetable) с вызывающим.
//
// fn    — функция потока; ОБЯЗАНА завершаться вызовом exit()
// arg   — передаётся в fn первым аргументом (через a0)
// stack — буфер под стек размером STACK_SIZE, выделенный вызывающим;
//        sp устанавливается на stack+STACK_SIZE с выравниванием по 16
//
// Возврат: tid (>0) вызывающему; -1 при ошибке.
// Новый поток наследует tgid вызывающего.
int clone(void (*fn)(void *), void *arg, void *stack);
```

Создавать потоки мы теперь умеем — по крайней мере на бумаге. Следующий вопрос сложнее: как им корректно умирать. Что означает exit при общей pagetable, кто и когда её всё-таки освобождает, и что такое ZOMBIE для потока.

4. Завершение потока: exit, ZOMBIE и вопрос «кто чистит»

Создание потока — это половина жизненного цикла, причём простая. Завершение сложнее, потому что в нём сходятся два неудобных факта:

1. поток не может освободить ресурсы, на которых стоит;
2. поток делит pagetable с другими — освобождать её можно только когда она никому больше не нужна.

Разберём оба.

1. Почему поток не убирает за собой сам

Посмотрим, какие ресурсы занимает завершающийся поток и кто их использует в момент exit():

Ресурс	Используется в момент exit?
пользовательский стек	уже нет — мы в ядре, работаем на kstack
kstack	да — прямо сейчас! exit выполняется на нём
trapframe	да — понадобится, если по пути случится trap
запись struct proc	да — планировщику нужен state, context
pagetable	да — она отображена, и trampoline живёт в ней

Ключевая строка — kstack. Функция exit() — это код ядра, выполняющийся на ядерном стеке завершающегося потока: локальные переменные exit, адреса возврата — всё лежит там. Освободить kstack изнутри exit — всё равно что выпилить сук, на котором сидишь: следующая же операция со стеком обратится к освобождённой памяти.

Отсюда универсальное правило, одинаковое для xv6, Linux и Windows:

Завершающийся поток освобождает всё, что может, помечает себя «завершён» — и отдаёт процессор навсегда. Последние ресурсы (kstack, trapframe, запись proc) освобождает кто-то другой, позже.

Состояние «я закончил, но мои останки ещё не убраны» и есть ZOMBIE.

4.1. Как это уже устроено в xv6: exit + wait

Хорошая новость: механика «умереть и быть убраным» в xv6 уже есть — для процессов. Посмотрим на неё, потому что thread_join будет её почти точной копией.

exit(status) (kernel/proc.c), по шагам:

```
void
exit(int status)
{
    struct proc *p = myproc();
    // 1. закрыть открытые файлы, отпустить cwd
    for(int fd = 0; fd < NOFILE; fd++){ ... fclose(f); ... }
    ...
    // 2. передать своих детей процессу init (reparent)
    reparent(p);

    // 3. разбудить родителя: вдруг он уже в wait()
    wakeup(p->parent);

    // 4. записать код выхода, стать зомби
    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // 5. уйти в планировщик НАВСЕГДА
    sched();
    panic("zombie exit"); // сюда не возвращаются
}
```

Заметь, чего exit не делает: не трогает pagetable, kstack, trapframe, не переводит запись в UNUSED. Он лишь закрывает то, что безопасно закрыть (файлы), выставляет ZOMBIE и исчезает в sched() — навсегда, потому что планировщик никогда не выберет ZOMBIE для запуска. panic после sched — страховка: если мы каким-то чудом вернулись, ядро сломано.

wait(addr) — вторая половина, сторона родителя:

```
int
wait(uint64 addr)
{
    ...
    for(;;){
        // перебор таблицы: есть ли у меня дети-зомби?
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                ...
                if(pp->state == ZOMBIE){
                    pid = pp->pid;
                    // забрать код выхода, СВОБОДИТЬ останки
                    copyout(..., &pp->xstate, ...);
                    freeproc(pp); // ← вот кто чистит!
                    ...
                    return pid;
                }
            }
        }
        // детей-зомби нет — уснуть до чьего-нибудь exit
        sleep(p, &wait_lock);
    }
}
```

Вот и ответ на вопрос «кто чистит»: freeproc вызывает родитель из wait, когда обнаруживает ребёнка-зомби. Родитель в этот момент стоит на *своём* kstack — освобождать чужой ему

ничто не мешает. А пара `wakeup(p->parent)` в `exit / sleep(p, &wait_lock)` в `wait` — тот самый механизм сна из раздела 1.3, который избавляет родителя от активного опроса таблицы.

Что ломается для потоков: общая `pagetable`

Теперь главная проблема лабораторной. Смотрим внутрь `freeproc`:

```
static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);    // ← ловушка!
    p->pagetable = 0;
    ...
    p->state = UNUSED;
}
```

`proc_freepagetable` освобождает таблицу страниц и всю пользовательскую память от 0 до `sz`. Для процесса — правильно. Для потока — катастрофа: `pagetable` общая, и её освобождение мгновенно убивает память всех остальных потоков группы, включая всё ещё работающие. Они продолжают выполняться и на первом же обращении к памяти... получают не ошибку даже, а произвольные данные: их страницы уже вернулись в аллокатор и могли быть выданы кому-то ещё. Это худший тип бага — тихая порча с отложенными симптомами.

Значит, `freeproc` должен различать: моя таблица или общая? Для этого мы ещё в разделе 1.4 завели флаг:

```
static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->pagetable && !p->shared_pagetable)    // ← правка
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    ...
}
```

Поток (`shared_pagetable == 1`) отдаёт `trapframe`, `kstack`, запись `proc` — но таблицу не трогает.

А кто тогда освободит `pagetable` вообще?

Если все потоки помечены `shared` и никто таблицу не освобождает — она утечёт вместе со всей памятью процесса. Нужна политика. Есть два честных варианта и один прагматичный:

Вариант А (refcount): счётчик потоков на `pagetable`; последний выходящий гасит свет. Красиво, симметрично, но требует аккуратного атомарного счётчика и порождает тонкий вопрос: «последним» может оказаться не главный поток, а произвольный — и тогда главный поток остаётся жить... на освобождённой таблице? Придётся дополнительно определять семантику.

Вариант Б (главный владеет): только создатель группы (тот, у кого `pid == tgid`, — он же имеет `shared_pagetable == 0`, ведь таблицу создал `fork`) освобождает таблицу своим обычным `freeproc`. Потоки — никогда.

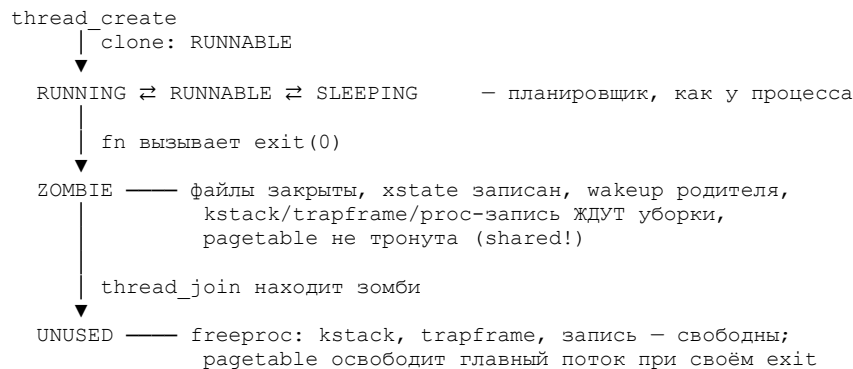
Выбираем Б: он получается *автоматически* из уже сделанной правки. У главного потока флаг `shared` не выставлен — его `freeproc` работает по-старому. У порождённых — выставлен, они таблицу пропускают. Ни строчки дополнительного кода.

Цена — контракт для пользователя:

Главный поток обязан пережить свои потоки: перед собственным `exit` он должен сделать `thread_join` всем созданным. Завершение главного потока при живых рабочих — неопределённое поведение.

Это ограничение мягче, чем кажется: ровно так пишут и корректные pthreads-программы (`main` делает `join` перед возвратом). Отличие настоящего Linux — там `exit` главного потока не убивает группу благодаря `refcount` на `mm_struct` — фиксируем в таблицу упрощений из 1.5. Вариант А остаётся дополнительным заданием для желающих.

Итог: жизненный цикл потока целиком



Осталась последняя недостающая деталь картинки: сам `thread_join` — как родителю дожидаться конкретного потока и не разбудиться зря. Это раздел 1.7, и он короткий: почти всё уже готово внутри `wait`.

4.2. Ожидание потока: `thread_join`

Раздел 1.6 закончился тем, что зомби-поток ждёт уборщика. Уборщик — это `thread_join`:

```
int thread_join(int tid);
// Ждёт завершения потока tid из своей группы.
// Возврат: tid при успехе, -1 при ошибке (нет такого потока / не наш).
```

Хорошая новость: писать почти нечего. Вся механика — перебор таблицы, сон, уборка — уже разобрана на примере `wait()`. Плохая новость: бездумно скопировать `wait` нельзя — семантика отличается в трёх местах. Разберём отличия, они и составляют содержание раздела.

Отличие 1: ждём конкретный `tid`, а не «любого ребёнка»

`wait()` отвечает на вопрос «завершился ли *хоть кто-нибудь* из моих детей?». `thread_join(tid)` — на вопрос «завершился ли *вот этот*?». Это упрощает цикл: вместо перебора всех детей с проверкой каждого — поиск одной записи:

```

int
thread_join(int tid)
{
    struct proc *pp;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        int found = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            acquire(&pp->lock);
            if(pp->pid == tid && pp->tgid == p->tgid && pp != p){ // наш поток?
                found = 1;
                if(pp->state == ZOMBIE){
                    freeproc(pp); // уборка (pagetable не тронет — 1.6)
                    release(&pp->lock);
                    release(&wait_lock);
                    return tid;
                }
            }
            release(&pp->lock);
        }

        if(!found){ // нет такого потока — ошибка
            release(&wait_lock);
            return -1;
        }

        sleep((void*)&proc, &wait_lock); // поток жив — уснуть до чьего-то exit
    }
}

```

Проверка `found` важна: без неё `join` несуществующего `tid` уснул бы навсегда. А проверка `tgid` — граница безопасности: нельзя заджойнить (и освободить!) чужой процесс, случайно угадав его `pid`.

Отличие 2: кого будит `exit` — проблема адресата

В паре `exit/wait` канал сна — адрес родителя: `exit` делает `wakeup(p->parent)`, `wait` спит на `sleep(p, ...)`, где `p` — сам родитель. Для процессов это работает, потому что `wait` зовёт только родитель.

С потоками жёстче: `join` может позвать кто угодно из группы, не только тот, кто делал `clone`. Поток А создал В и С; С делает `thread_join(B)`. Но `exit` потока В разбудит `B->parent` — то есть А, который вовсе не ждёт. А С, который ждёт, спит на *своём* адресе и не проснётся никогда.

Минимальное решение — сохранить контракт «`join` зовёт создатель» (тогда `parent`-механика работает как есть) и зафиксировать это ограничением. Но есть решение не сложнее и без ограничения: будить всех потенциальных ожидающих. Меняем адресата в `exit`:

```

// в exit(), рядом с wakeup(p->parent):
wakeup(p->parent); // процессная семантика (wait) — оставить
wakeup((void*)&proc); // + общий канал "кто-то завершился" для join
а join спит на том же общем канале:
с
sleep((void*)&proc, &wait_lock); // вместо sleep(p, ...)

```

Адрес `&proc` (сам массив таблицы) здесь — просто уникальное число, известное всем: удобный общий канал. Просыпаться будут и те, кто ждал другой `tid`, — но вспомни устройство цикла: проснувшийся перепроверяет условие и, не найдя свой зомби, засыпает обратно. Ложное пробуждение — это лишний оборот цикла, а не ошибка.

Плата — эффективность: при 50 потоках, ждущих разные `tid`, каждый `exit` будит всех 50, из которых 49 повернутся на другой бок. Это классика под названием *thundering herd*, и мы оставляем её в коде сознательно: лабораторная №2 (*futex*) начнётся ровно с вопроса «как

будить только того, кого надо». Пусть проблема сначала заведётся — тогда её решение будет мотивированным.

Отличие 3: freeproc снова, и почему он уже готов

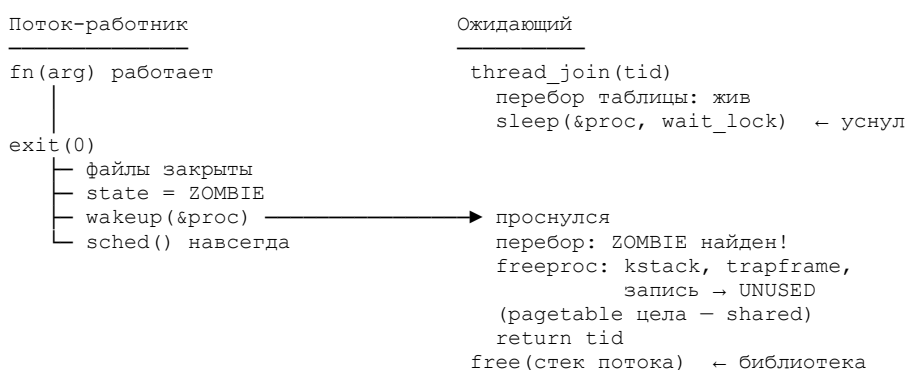
Внутри `join` мы зовём тот же `freeproc`, что и `wait`. Никакой отдельной «поточной» версии не нужно: правка с `shared_pagetable` из раздела 1.6 уже делает его корректным для обоих случаев — процессный зомби потеряет `pagetable`, поточный сохранит её для группы. Одна правка обслужила оба пути уборки; это признак того, что флаг был поставлен в правильном месте.

Стек потока (пользовательский, который выделял `thread_create` через `malloc`) `freeproc`, разумеется, не трогает — ядро о нём вообще не знает, это просто байты в общей куче. Вернуть его должна библиотека:

```
// userspace, user/uthread.c - набросок
int
thread_join_t(int tid)
{
    int r = thread_join(tid);           // syscall
    if(r > 0)
        free(stack_of(tid));           // вернуть стек в кучу
    return r;
}
```

— что требует помнить соответствие `tid` → стек (таблица в библиотеке). В базовой версии допустимо и упрощение «стеки не переиспользуем» (утечка до конца процесса), но аккуратный вариант — дополнительное задание, и он честнее.

Собранный жизненный цикл: обе стороны



5. Замер времени в xv6

Практическая часть потребует измерять: сколько стоит clone против fork, как долго работал поток. Разберёмся с инструментами заранее — в xv6 их два, и первый почти бесполезен.

ticks: то, что есть из коробки

Ядро ведёт глобальный счётчик `ticks`, увеличивая его в обработчике таймерного прерывания. Интервал задан в `kernel/trap.c` (в старых ревизиях — `start.c`):

```
// ask for the next timer interrupt ... 1000000 is about a tenth
// of a second.
w stimecmp(r time() + 1000000);
```

Один тик ≈ 100 миллисекунд. Из userspace счётчик доступен через syscall uptime(). Для наших задач это катастрофически грубо: создание потока занимает сотни *микросекунд* —

между двумя вызовами `uptime()` пройдут тысячи таких операций, и разность почти всегда будет нулём. Годится только для операций длительностью в секунды.

rdtime: аппаратный счётчик RISC-V

У RISC-V есть CSR (control and status register) time — счётчик реального времени, инкрементируемый с фиксированной частотой независимо от прерываний. Читается одной инструкцией `rdtime`, причём прямо из user mode — без системного вызова:

```
// user/timing.h
#ifndef TIMING_H
#define TIMING_H

// RISC-V rdtime: счётчик реального времени.
// В QEMU virt частота 10 МГц: 1 тик = 100 нс.
static inline uint64
r_time(void)
{
    uint64 x;
    asm volatile("rdtime %0" : "=r" (x));
    return x;
}

#define TICKS_PER_US 10
#define TICKS_PER_MS 10000

#endif
```

В машине QEMU virt частота счётчика — 10 МГц: один тик = 100 нс. Это в миллион раз точнее `ticks`, а стоит одну инструкцию. Основной инструмент всех наших замеров.

Ловушка: счётчик нужно разрешить

Наивная попытка использовать `rdtime` из user-программы встретит вот такую панику (реальный вывод с эталонной реализации):

```
$ benchtest
usertrap(): unexpected scause 0x2 pid=15
sepc=0xc01029f3 stval=0xc01029f3
```

`scause 0x2` — illegal instruction. Разбор улик: `sepc` указывает на инструкцию в нашей программе, а `stval` содержит... её код. Дизассемблируем бинарник по адресу `sepc`:

```
1022: c01029f3    rdtime    s3
```

`stval == 0xc01029f3 == код`

`rdtime` — процессор отверг саму инструкцию. Причина: доступ к счётчикам из непривилегированных режимов управляется регистрами `mcounteren` (M-mode разрешает S-mode) и `scounteren` (S-mode разрешает U-mode), и по умолчанию U-mode доступа не имеет. Ядро xv6 само читает `time` без препятствий (оно в S-mode, и M-mode-настройка это покрывает), а вот пользователю нужно разрешить явно.

Правка из двух строк. В `kernel/start.c`, в `start()` (M-mode, рядом с настройкой делегирования):

```
// разрешить чтение счётчиков cycle/time/instrret из S-mode
w_mcounteren(0x7);
```

В `kernel/main.c`, в `main()` — так, чтобы выполнил каждый hart (и нулевой, и остальные):

```
// разрешить чтение счётчиков из U-mode (для rdtime в user-программах)
w_scounteren(0x7);
```

Обёртки, если их нет в `kernel/riscv.h`, — по образцу соседних:

```
static inline void
w_mcounteren(uint64 x)
{
    asm volatile("csrw mcounteren, %0" : : "r" (x));
}

static inline void
w_scounteren(uint64 x)
{
    asm volatile("csrw scounteren, %0" : : "r" (x));
}
```

Проверка: программа, печатающая `r_time()` дважды с `sleep(10)` между вызовами; разность должна составить около $10 \text{ тиков} \times 100 \text{ мс} \times 10^7 \text{ тиков/с} = 10^7 \text{ тиков rdtime}$.

Методика: как мерить, чтобы числам можно было верить

QEMU — эмулятор: абсолютные времена зависят от нагрузки хост-машины и режима трансляции, от запуска к запуску плывут. Четыре правила снижают шум до приемлемого:

1. Мерить агрегат, а не единичную операцию. Одна операция тонет в шуме; тысяча — усредняет его:

```
uint64 t0 = r_time();
for(int i = 0; i < N; i++){
    int tid = thread_create(empty_thread, 0);
    thread_join_t(tid);
}
uint64 t1 = r_time();
printf("avg: %lu us\n", (t1 - t0) / TICKS_PER_US / N);
```

2. Повторять и сравнивать прогоны. Первый запуск дополнительно платит за холодные кэши; ориентироваться стоит на устоявшиеся значения. Реальные данные эталонной реализации — два прогона `benchtest`:

```
clone+join: 378 us each      fork+wait:  982 us each
clone+join: 380 us each      fork+wait: 1198 us each
```

`clone` стабилен (378/380), `fork` гуляет на 20% — но соотношение $\sim 3\times$ воспроизводится уверенно.

3. Сравнивать соотношения, а не абсолюты. «`clone` втрое быстрее `fork`» переживёт смену хост-машины; «380 мкс» — нет.

4. Замер — в одном потоке. `t0` и `t1` берёт `main`: создать всех $\rightarrow$ join всех $\rightarrow$ разность. Это `wall-clock` всей параллельной фазы; попытки мерить «внутри» потоков сложат несравнимые интервалы.

Сводка

Инструмент	Гранулярность	Syscall	Применение
<code>uptime()</code> / <code>ticks</code>	~ 100 мс	да	только очень длинные интервалы
<code>rdtime</code>	100 нс	нет	все замеры лабораторной
счётчики событий	точно	—	сравнение алгоритмов (пригодится в лаб. 2–3)

Третья строка — на вырост: в следующих лабораторных мы будем сравнивать реализации не временем, а числом событий (сколько раз ушли в `syscall`, сколько раз заснули) — в эмуляторе такие метрики честнее секундомера. Теория на этом закрыта; переходим к практике.

6. Два предупреждения перед практикой

Прежде чем запускать первые потоки, нужно откалибровать ожидания. Два факта о многопоточном мире, без которых результаты экспериментов будут выглядеть как поломки, а настоящие поломки — как результаты.

6.1. Планировщик не обещает ничего

Вопрос на интуицию: программа создаёт потоки А, потом В, оба печатают своё имя. Что появится на экране раньше?

Интуиция говорит «А — его же создали первым». Правильный ответ: **неизвестно**. Может А, может В, может даже В успеет завершиться до того, как А получит первый квант. С момента, когда clone выставил state = RUNNABLE, поток — собственность планировщика, а планировщик не гарантирует *ничего*: ни кто стартует первым, ни в каком порядке потоки будут перемежаться, ни сколько инструкций каждый выполнит до переключения. На результат влияют число CPU, таймерные прерывания, чем заняты другие процессы — факторы вне контроля программы.

Из этого следует определение, которое стоит выучить как формулу:

Корректная многопоточная программа обязана быть корректной при любом допустимом чередовании выполнения потоков.

«У меня пять раз подряд напечаталось в правильном порядке» — не аргумент корректности: это утверждение о пяти чередованиях из астрономического числа возможных. Если правильность программы зависит от того, кто выполнится первым, — программа уже неправильная, просто ошибка пока не проявилась. Единственный легальный способ повлиять на порядок — явная синхронизация (join уже даёт её зачаток: «после join(t) поток t гарантированно завершён»; остальной арсенал — лабораторные 2–5).

Практическое следствие для экспериментов: перемешанный, меняющийся от запуска к запуску вывод в Э1 — это норма и ожидаемый результат, а не дефект реализации.

6.2. Гейзенбаги: почему многопоточные ошибки не ловятся привычными способами

Ошибки из однопоточного мира воспроизводимы: тот же ввод — то же падение, можно итеративно сужать. Ошибки синхронизации устроены иначе: они проявляются только на *некоторых* чередованиях. Программа с гонкой может пройти тысячу запусков и упасть на тысяча первом — на другой машине, под другой нагрузкой, у преподавателя на защите.

Хуже того: попытка посмотреть на баг его меняет. Вставили printf для отладки — printf делает syscall, syscall — это трапы и переключения, тайминги сдвинулись, чередование стало другим — баг исчез. Убрали printf — вернулся. За квантовомеханическую аналогию (наблюдение меняет наблюдаемое) такие ошибки прозвали **гейзенбагами**. Тот же эффект дают отладчик (остановка в gdb радикально меняет расписание) и оптимизации компилятора.

Реальный пример из эталонной реализации этой лабораторной. Гонка на счётчике (эксперимент Э2, два потока инкрементируют общую переменную):

Итераций на поток	Результат
1 000 000	потерь нет , шесть запусков подряд

20 000 000	потерь нет, шесть запусков подряд
100 000 000	потеряно 36%, 30%, 22% — в трёх запусках

Гонка была в коде всегда — с первой итерации. Но при коротком цикле первый поток успевал отработать целиком до того, как планировщик запускал второй: чередования не возникало, дефект молчал. Двенадцать «чистых» прогонов не доказали ровным счётом ничего. Отсюда рабочие правила:

- **отсутствие симптома $\neq$ отсутствие бага** — тестирование показывает наличие гонок, но не их отсутствие;
- **гонку нужно уметь провоцировать**: длинные циклы, много потоков, реальная параллельность — сужение окна уязвимости прячет ошибку, расширение — проявляет;
- **воспроизводимость измеряется сериями**: результат гонки — распределение по многим запускам, а не число из одного;
- **printf-отладка для таких багов ненадёжна by design**; честные инструменты — инварианты в коде, счётчики событий, анализ по построению («могут ли два потока оказаться здесь одновременно?»).

И последний штрих — направление ошибки. В Э2 результат всегда *меньше* правильного, никогда не больше. Это не случайность: counter++ — это три инструкции (загрузить, увеличить, сохранить), и единственное, что может сделать неудачное чередование, — заставить сохранение записать устаревшее значение поверх чужого свежего, стерев чужие инкременты. Добавить лишних оно не способно. Умение вывести направление ошибки из механики чередования — маленький образец главного навыка курса: рассуждать о том, что могло произойти, а не только о том, что произошло.

7. Практическая часть

Что мы строим

К концу практики в вашем xv6 появится API потоков:

```
int thread_create(void (*fn)(void *), void *arg); // создать; -> tid или -1
int thread_join_t(int tid);                       // дождаться и убрать
void thread_exit(void);                           // завершить текущий
```

и три тестовые программы. Целевой результат выглядит так (фрагмент реального вывода):

```
$ threadtest
main: tid 4
thread 0 (titdth h5t): rhrreelada id3n ea d2 e (t0 (1...
...
main: all joined
$ racetest
expected 200000000, got 127378838, lost 72621162
$ racetest a
expected 200000000, got 200000000, lost 0
```

Перемешанные до нечитаемости строки и «потерянные» 36% инкрементов — не поломки, а ожидаемые результаты: это гонки из разделов 1.1 и 1.9, воспроизведённые на вашей собственной реализации. Если вы получили такое — вы построили настоящие потоки.

Путь разбит на шесть шагов. Каждый заканчивается чекпойнтом — проверкой, которую нужно пройти прежде чем идти дальше. Не пропускайте чекпойнты: почти все ошибки следующих шагов диагностируются через работоспособность предыдущих.

Шаг 1. Инфраструктура замера времени

Начинаем не с потоков, а с секундомера — он понадобится и для финальных бенчмарков, и, что важнее, для отладки по дороге. Теория — раздел 1.8; здесь только действия.

1.1. Заголовок timing.h

Создайте user/timing.h:

```
#ifndef TIMING_H
#define TIMING_H

// RISC-V rdtm: счётчик реального времени.
// В QEMU virt частота 10 МГц: 1 тик = 100 нс.
static inline uint64
r_time(void)
{
    uint64 x;
    asm volatile("rdtime %0" : "=r" (x));
    return x;
}

#define TICKS_PER_US 10
#define TICKS_PER_MS 10000

#endif
```

1.2. Разрешение счётчиков

Без этого пункта первая же программа с r_time() умрёт с illegal instruction (разбор — в 1.8). В kernel/riscv.h добавьте рядом с другими обёртками w_*:

```
static inline void
w_mcounteren(uint64 x)
{
    asm volatile("csrw mcounteren, %0" : : "r" (x));
}

static inline void
w_scounteren(uint64 x)
{
    asm volatile("csrw scounteren, %0" : : "r" (x));
}
```

В kernel/start.c, в функции start():

```
// разрешить чтение счётчиков cycle/time/instrret из S-mode
w_mcounteren(0x7);
```

В kernel/main.c, в main() — в путь, который проходит каждый hart (и нулевой, и остальные — иначе rdttime будет работать или падать в зависимости от того, на каком CPU оказался процесс):

```
// разрешить чтение счётчиков из U-mode
w_scounteren(0x7);
```

1.3. Тестовая программа

user/timetest.c:

```
#include "kernel/types.h"
#include "user/user.h"
#include "user/timing.h"

int
main(void)
{
    uint64 t0 = r_time();
    pause(10); // 10 тиков ядра = ~1 секунда
    uint64 t1 = r_time();

    printf("rdtime delta: %lu (expected ~100000000)\n", t1 - t0);
    printf("          = %lu ms\n", (t1 - t0) / TICKS_PER_MS);
    exit(0);
}
```

Добавьте \$U/_timetest\ в список UPROGS в Makefile.

Чекпоинт 1

```
$ timetest
rdtime delta: 10023481 (expected ~100000000)
              = 1002 ms
```

Разность около 10^7 (10 тиков $\times$ 100 мс $\times$ 10 МГц), миллисекунды около 1000. Отклонение в единицы процентов — норма (сон гарантирует «не меньше», см. 1.8).

Если вместо этого:

```
usertrap(): unexpected scause 0x2 pid=N
           sepc=0x... stval=0xc01029f3
```

— illegal instruction, причём stval равен коду инструкции rdttime: не сработал пункт 1.2. Проверьте, что scounteren выставляется на **всех** hart'ах, и пересоберите полностью (make clean && make qemu).

Дисциплина пересборки. Возьмите за правило после правок в riscv.h, memlayout.h, Makefile или user.ld собирать через make clean. Система зависимостей xv6 не отслеживает часть таких файлов, и инкрементальная сборка может оставить в fs.img старые бинарники — вы будете отлаживать код, которого уже нет в исходниках. Это самая дорогая по времени категория «ошибок» в лабораторной, потому что ошибки-то и нет.

Шаг 2. Скелет системного вызова clone

Прежде чем писать логику — проложим трубопровод: `syscall`, который можно вызвать из user-программы и получить ответ. Заглушка вернёт `-1`; содержательной она станет в Шаге 3.

Регистрация системного вызова в `xv6` затрагивает шесть файлов. Номера ниже — примерные: **продолжайте свою таблицу** в `syscall.h` (посмотрите последний занятый номер; в разных ревизиях он различается).

`kernel/syscall.h`:

```
#define SYS_clone      23    // подставьте свои свободные номера
#define SYS_thread_join 24
#define SYS_gettid     25
```

`kernel/syscall.c` — внешние объявления и записи в таблице `syscalls[]`:

```
extern uint64 sys_clone(void);
extern uint64 sys_thread_join(void);
extern uint64 sys_gettid(void);
...
[SYS_clone]      sys_clone,
[SYS_thread_join] sys_thread_join,
[SYS_gettid]     sys_gettid,
```

`kernel/sysproc.c` — обёртки, распаковывающие аргументы:

```
uint64
sys_clone(void)
{
    uint64 fn, arg, stack;
    argaddr(0, &fn);
    argaddr(1, &arg);
    argaddr(2, &stack);
    return clone(fn, arg, stack);
}
```

```
uint64
sys_thread_join(void)
{
    int tid;
    argint(0, &tid);
    return thread_join(tid);
}
```

```
uint64
sys_gettid(void)
{
    return myproc()->pid;
}
```

`kernel/defs.h` — прототипы для `proc.c`:

```
int      clone(uint64, uint64, uint64);
int      thread_join(int);
```

`kernel/proc.c` — заглушки (настоящие тела — Шаги 3 и 5):

```
int
clone(uint64 fn, uint64 arg, uint64 stack)
{
    return -1;
}
```

```
int
thread_join(int tid)
{
    return -1;
}
```

`user/usys.pl` — генерация `userspace`-заглушек:

```
entry("clone");
entry("thread_join");
entry("gettid");
```

user/user.h — прототипы для программ:

```
int clone(void (*fn)(void*), void *arg, void *stack);
int thread_join(int tid);
int gettid(void);
```

Чекпоинт 2

Мини-программа: вызвать `gettid()` и `clone(0, 0, 0)`, напечатать результаты. Ожидаемо: `gettid` возвращает `pid` (совпадает с `getpid()`), `clone` возвращает `-1`. Трубопровод работает — обе стороны договорились о номерах.

Классический сбой на этом шаге — рассинхрон номеров: `usys.pl` генерирует заглушки из `syscall.h` на момент сборки; если после правки номеров собрать без `clean`, юзерская заглушка может звать старый номер — и «`clone`» молча выполнит чужой `syscall`. Симптомы бывают фантастическими; лечение одно — `make clean`.

Сверка с эталоном: те же эксперименты на Linux

Вы построили потоки по образу Linux (раздел 1.2) — пришло время очной ставки. Идея раздела: прогнать те же три эксперимента на настоящем `pthread` и убедиться, что ваша реализация воспроизводит не только API, но и поведение — вплоть до формы гонки. Заодно увидите инструменты наблюдения, которых в `xv6` нет.

Работаем на хост-машине (Linux, `gcc`); `xv6` не нужен.

Э1 на pthreads

`threadtest_linux.c` — перевод `threadtest` на `pthread` почти дословный:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

void *worker(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < 10; i++)
        printf("thread %ld (tid %ld): line %d\n",
            id, syscall(SYS_gettid), i);
    return NULL;
}

int main(void) {
    pthread_t t[4];
    printf("main: pid %d\n", getpid());

    for (long i = 0; i < 4; i++)
        pthread_create(&t[i], NULL, worker, (void *)i);
    for (int i = 0; i < 4; i++)
        pthread_join(t[i], NULL);

    printf("main: all joined\n");
    return 0;
}
```

```
gcc threadtest_linux.c -o threadtest_linux -pthread
./threadtest_linux
```

Сравните с `xv6`-выводом. Сходство: порядок строк непредсказуем и меняется между запусками — планировщик Linux обещает так же мало, как планировщик `xv6`. Различие: строки перемешаны между собой, но не внутри — каши из букв нет. Причина: `glibc-printf` захватывает внутреннюю блокировку `stdout` на время вывода строки; это ровно тот `mutex`, которого нет в `printf xv6` (вопрос (б) из Э1 — вот и ответ, увиденный глазами).

Попутные отличия API, достойные строки в отчёте: `pthread_create` возвращает результат потока через указатель (у нас `retval` — доп. задание 6.4); `worker` завершается `return` —

pthreadс подставляет трамплин, делающий неявный pthread_exit (наш контракт «обязателен thread_exit») — раздел 1.5, решение 3).

Увидеть clone своими глазами

Главный аттракцион раздела. Ваша теория утверждала: pthread_create — это обёртка над clone() с набором флагов. Проверка:

```
strace -f ./threadtest_linux 2>&1 | grep clone
```

Ожидаемо (одна строка на каждый из четырёх потоков):

```
clone(child_stack=0x7f3a2b5fefb0,  
      flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD/  
          CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID/  
          CLONE_CHILD_CLEARPID, ...) = 12347
```

Разбор флагов — по таблице раздела 1.2, теперь напротив каждого можно ставить галочку «есть у нас»:

Флаг Linux	В нашей реализации
CLONE_VM	есть — общая pagetable (сердце Шага 3)
CLONE_THREAD	есть — tgid
CLONE_FS, CLONE_SIGHAND, CLONE_SYSVSEM	нет соответствующих ресурсов в xv6
CLONE_FILES	нет — у нас снимок таблицы, не разделение (1.3)
CLONE_SETTLS	нет — TLS не реализован (1.5, таблица упрощений)
CLONE_PARENT_SETTID CHILD_CLEARPID	нет — механизм для join через futex; наш join устроен иначе, и в лабораторной №2 вы поймёте, чем этот способ лучше

Три «нет» — это не недоделки, а осознанная граница базовой версии, и теперь она видна предметно: вот флаги, которые отделяют учебную реализацию от промышленной.

И второй инструмент — потоки глазами ядра:

```
$ ./threadtest_linux & # добавьте sleep(30) в main, чтобы уснуть  
$ ls /proc/$(pgrep threadtest_lin)/task/  
$ cat /proc/$(pgrep threadtest_lin)/task/*/comm
```

Пять tid'ов в task/ — main + четыре worker'a: тот самый TGID-контейнер из 1.2, потрогать который в xv6 можно только перебором таблицы процессов из ядра.

Э2 на pthreads: та же гонка, те же законы

Перевод racetest (замените thread_create/join на pthread-аналоги, N_ITER верните к 10–100M) и серия запусков:

```
gcc racetest_linux.c -o racetest_linux -pthread -O0 # -O0 важен: см. вопрос ниже  
for i in 1 2 3 4 5; do ./racetest_linux; done
```

Ожидаемая картина — знакомая: потери есть, каждый раз разные, результат всегда занижен. Форма гонки не зависит от ОС — она следует из тройки load/add/store и существования чередований, а это свойства архитектуры, не ядра. Скорее всего изменится масштаб: на реальном железе с настоящей параллельностью ядер потери обычно выше и порог проявления ниже, чем в QEMU, — сравните свои цифры и объясните направление разницы.

Вопрос со звёздочкой в отчёт: соберите с -O2 без volatile — что стало с потерями и почему?

Э3: цена потока на Linux

```
// bench_linux.c: N=100 циклов pthread_create+join против fork+wait
// (текст симметричен benchtest.c; таймер — clock_gettime(CLOCK_MONOTONIC))
```

Типичные результаты на современном железе: поток ~30–80 мкс, fork ~150–500 мкс — оба на порядок быстрее ваших xv6-цифр (QEMU-эмуляция!), но соотношение того же порядка: процесс в разы дороже. Причём fork Linux уже смягчён сору-on-write — копируются таблицы страниц, не память; в xv6 uvmсору честно копирует всё. Отсюда вопрос в отчёт: у кого разрыв clone/fork должен расти быстрее с размером образа — у xv6 или у Linux — и почему? (Свяжите с заданием 6.5.)

Сводная таблица для отчёта

Финальный артефакт раздела — таблица очной ставки, заполняется своими данными:

Наблюдение	xv6 (ваша реализация)	Linux (pthreads)	Совпало?
Порядок строк Э1	недетерминирован	недетерминирован	✓
Каша внутри строк	есть	нет (lock в glibc-printf)	✗ — объяснимо
Потери в гонке	есть, ~20–40%, разные	есть, свои %, разные	✓ по форме
Результат гонки	только занижен	только занижен	✓
Лечение атомиком	lost 0	lost 0	✓
clone vs fork	~3×	в разы	✓ по порядку
Флаги clone	VM, THREAD-аналог	+ FILES, SETTLS, TID- семейство	частично

Строки «совпало» — доказательство, что построено настоящее; строки «не совпало» — каждая с объяснением и адресом (какая лабораторная или доп. задание закрывает разрыв). Такая таблица — лучший финал отчёта: она показывает не «программа работает», а «я понимаю, что именно работает и где проходит граница».

6. Задания

6.1. Обязательная часть

1. Инфраструктура времени. `timing.h` + разрешение счётчиков; чекпоинт 1 пройден, вывод `timetest` — в отчёт.
2. Системный вызов `clone`. По спецификации раздела 1.5: общее адресное пространство, слоты `trapframe`, `sscratch`-схема трамплина. Критерий: `usertests -q` проходит целиком (регрессия!), `clonetest` из чекпоинта 4 работает.
3. `exit/thread_join` + библиотека `uthread`. Чекпоинты 5–6: `join` конкретного `tid`, возврат `-1` по несуществующему, стек возвращается в кучу, `threadtest` трижды подряд.
4. Эксперименты Э1–Э3. По методикам раздела 3: серии запусков, таблицы (включая таблицу подбора `N_ITER` из Э2 — она обязательна), объяснения через механизм.
5. Сверка с Linux. Раздел 4: `pthread`-версии Э1–Э2, вывод `strace` с разбором флагов, итоговая сводная таблица 4.5.
6. Отчёт. Дифф ядра (`git diff` от ванильного `xv6`), выводы программ копиями с экрана, ответы на вопросы «в отчёт» из разделов 3–4.

6.2. Дополнительное задание

Реализовать возможность возврата значения из потока. `void *`-результат: `thread_exit_val(void *retval) + thread_join_t(tid, void **retval)`. Подсказка: `xstate` уже переносит `int` через `ZOMBIE` — обобщите механизм. Обсудите в отчёте: где хранить `retval`, пока `join` не пришёл, и почему нельзя на стеке потока.

7. Контрольные вопросы

1. Покажите в диффе три содержательных отличия вашего `clone` от `fork`. Почему остальной код почти совпадает?
2. Зачем каждому потоку два стека? Покажите оба в коде: кто выделяет, где хранится указатель, кто освобождает.
3. Проследите путь аргумента `arg`: от вызова `thread_create` до первой инструкции `fn`. Через какие регистры и структуры он проходит? Почему именно `a0`?
4. Почему `exit` не может освободить `kstack` завершающегося потока? Кто и в какой момент это делает? Свяжите с назначением состояния `ZOMBIE`.
5. Ваш `thread_join` будит все ждущие потоки при каждом `exit`. Опишите сценарий, где это заметно дорого. Каким свойством должен обладать механизм, чинящий это?
6. Почему на `N_ITER=1M` гонка молчала, а на `100M` — проявилась? Оцените по вашим замерам `r_time`, сколько итераций цикла помещается в квант планировщика, и свяжите числа.
7. Ваши потоки и горутини `Go` — обе «единицы выполнения». Используя дополнительную литературу докажите, что горутини — не поток ОС, и объясните, почему `Go` тем не менее вызывает `clone()` — что именно он им создаёт?